Fast API

Projet : Création d'une API de Gestion de Citations avec FastAPI et Intégration Streamlit 2
Prérequis : 2
Introduction : 2
Création d'un dictionnaire Python : 2
Pydantic 3
Construction de l'API FastAPI : 3
Route racine (/) : 3
Route GET pour toutes les citations (/citations) : 3
Route GET pour une citation par ID (/citations/{citation_id}) : 4
Comment atteindre les routes GET dans le navigateur : 5
Création d'une page Streamlit simple : 5
Exemple API / Streamlit 6
API 6
Streamlit 6
Routes POST, PUT et DELETE dans l'API : 7
Ajout d'un formulaire Streamlit pour POST, PUT et DELETE 7

Projet : Création d'une API de Gestion de Citations avec FastAPI et Intégration Streamlit

Prérequis :

- Environnement virtuel `(.venv)` configuré.
- Fichier requirements.txt contenant les dépendances nécessaires (FastAPI, Uvicorn, Loguru, Pydantic, requests, streamlit).
- Fichier README.md décrivant l'installation et les routes de l'API.

Introduction :

Ce projet vous guidera pas à pas dans la création d'une API FastAPI simple pour gérer des citations. Nous apprendrons à construire des routes pour effectuer des opérations CRUD (Créer, Lire, Mettre à jour, Supprimer) et à intégrer cette API avec une application Streamlit pour une interface utilisateur interactive.

Création d'un dictionnaire Python :

Explication : Un dictionnaire Python est une structure de données qui stocke des paires clé-valeur. Les clés sont uniques et les valeurs peuvent être de n'importe quel type. Nous allons utiliser un dictionnaire pour simuler une base de données de citations.

Créez un dictionnaire nommé citations et remplissez-le avec quelques citations. Chaque citation aura un ID unique comme clé.

```
citations = {
    1: "La vie est un mystère qu'il faut vivre, et non un problème à résoudre.",
    2: "Le plus grand plaisir de la vie est l'amour.",
    3: "L'avenir appartient à ceux qui croient à la beauté de leurs rêves.",
}
```

Pydantic

L'utilisation de Pydantic nous permet de définir des modèles de données clairs et de valider les données entrantes. Par exemple, pour les routes POST et PUT, nous avons créé un modèle Citation avec un champ citation de type str. Pydantic garantit que les requêtes reçues contiennent bien une donnée au format demandé, simplifiant ainsi la gestion et la validation des données. Si une requête ne respecte pas le modèle défini (par exemple, si le champ citation est absent ou d'un type incorrect), Pydantic génère automatiquement une erreur HTTP, assurant ainsi la cohérence et la fiabilité des données traitées par l'API. Cette approche réduit considérablement la quantité de code de validation manuelle nécessaire, rendant notre API plus robuste et facile à maintenir.

Construction de l'API FastAPI :

Initialiser une application FastAPI.

```
from fastapi import FastAPI, HTTPException
app = FastAPI()
```

Route racine (/) :

Créez une route GET à la racine de l'API qui renvoie un message de bienvenue ("Hello World!").

```
@app.get("/")
async def root():
    logger.info("Route '/' appelée.")
    return {"message": "Hello World!"}
```

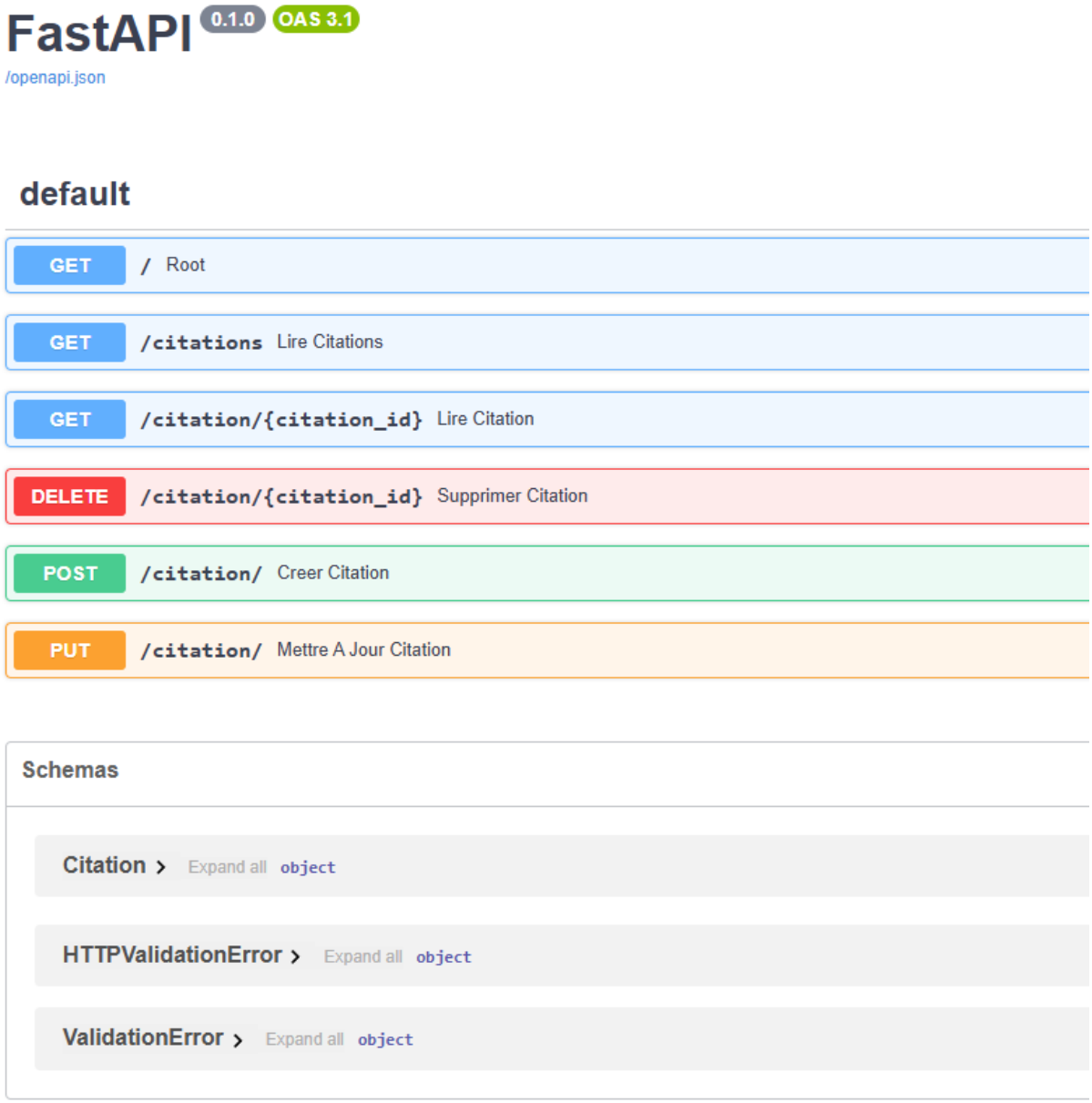
Route GET pour toutes les citations (/citations) :

Créez une route GET qui renvoie toutes les citations stockées dans le dictionnaire.

```
@app.get("/citations")
async def lire_citations():
    return citations
```

Route GET pour une citation par ID (`/citations/{citation_id}`) :

- Créez une route GET qui renvoie une citation spécifique en fonction de son ID.
- Gérez les erreurs si l'ID de la citation n'existe pas.
- Explication des routes `/docs` et `/redocs` de FastAPI :
    - FastAPI génère automatiquement une documentation interactive pour votre API aux routes `/docs` et `/redocs`.
    - `/docs` utilise Swagger UI et `/redocs` utilise ReDoc.



Citation ci dessus est un model Pydantic

Comment atteindre les routes GET dans le navigateur :

Lancez l'API avec Uvicorn (`uvicorn main:app --host 127.0.0.1 --port 9000 --reload`). Ouvrez votre navigateur et accédez à `http://127.0.0.1:8000/citation` pour toutes les citations, et à `http://127.0.0.1:8000/citation/1` pour la citation avec l'ID 1.

```
@app.get("/citation/{citation_id}")
async def lire_citation(citation_id: int) -> Dict[int, str]:
    logger.info(f"Route '/citation/{citation_id}' appelée.")
    if citation_id in citations:
        logger.debug(f"Citation trouvée : {citations[citation_id]}")
        return {citation_id: citations[citation_id]}
    else:
        logger.warning(f"Citation {citation_id} non trouvée.")
        raise HTTPException(status_code=404, detail="Citation non
trouvée")
```

Création d'une page Streamlit simple :

Pour communiquer entre une application Streamlit et une API FastAPI, nous utilisons la bibliothèque requests afin d'effectuer des requêtes HTTP. Streamlit, agissant comme le client, envoie des requêtes à l'API FastAPI, qui joue le rôle de serveur. Pour ce faire, nous définissons l'URL de l'API et utilisons les fonctions de requests telles que get, post, put ou delete, en fonction de l'opération souhaitée. Pour les requêtes avec des données, comme les POST ou PUT, nous envoyons les données au format JSON. Une fois la requête envoyée, Streamlit reçoit la réponse de l'API. Cette réponse contient un code de statut HTTP indiquant le succès ou l'échec de la requête, ainsi que les données renvoyées par l'API, généralement au format JSON. Streamlit traite ensuite ces données et les affiche à l'utilisateur, créant ainsi une interaction dynamique entre l'interface utilisateur et l'API.

Créez une application Streamlit qui se connecte à l'API:

- Affichez toutes les citations renvoyées par la route GET `/citation`.
- Affichez le nombre total de citations.
- Ajoutez un sélecteur déroulant qui permet de choisir une citation par ID et d'afficher son contenu.

Exemple API / Streamlit

API

```
class Citation(BaseModel):
    citation: str

@app.post("/citation/")
async def creer_citation(citation: Citation) -> Dict[str, str]:
    logger.info("Route '/citation/' (POST) appelée.")
    nouvel_id = max(citations.keys()) + 1
    citations[nouvel_id] = citation.citation
    logger.debug(f"Citation ajoutée : {citation.citation}")
    return {"message": f"Citation ajoutée avec l'ID {nouvel_id}"}
```

Streamlit

```
# Fonction pour ajouter une citation
def add_citation(citation):
  response = requests.post(f"{API_URL}/citation/", json={"citation":
citation})
  return response.json()

st.header("Ajouter une Citation")
with st.form("add_citation_form"):
  citation = st.text_area("Entrez la nouvelle citation")
  submitted = st.form_submit_button("Ajouter")
  if submitted:
    result = add_citation(citation)
    st.write(result)
```

Routes POST, PUT et DELETE dans l'API :

Créez une route POST /citation/ pour ajouter une nouvelle citation.

```
    nouvel_id = max(citations.keys()) + 1

    citations[nouvel_id] = citation.citation
```

Créez une route PUT /citation/{citation_id} factice qui modifiera une citation.

```
citations[citation_id] = citation.citation
```

Créez une route DELETE /citation/{citation_id} qui sera factice et prétendra d'avoir supprimé une citation.

```
return {"message": f"Citation {citation_id} supprimée"}
```

Ajout d'un formulaire Streamlit pour POST, PUT et DELETE

Ajoutez un formulaire à l'application Streamlit qui permet à l'utilisateur de saisir une nouvelle citation et de l'envoyer à la route POST de l'API.

Ajoutez des boutons dans l'application Streamlit pour les opérations PUT et DELETE.

Formatage des réponses

On peut utiliser le module Response pour formater les réponses

```
from fastapi import FastAPI
from fastapi.responses import Response
import json

app = FastAPI()

@app.get("/formatted")

def formatted():
    data = {"name": "Wojo", "age": 30, "interests": ["LoL", "Python"]}
```

```
    json_str = json.dumps(data, indent=2, ensure_ascii=False)
    return Response(content=json_str, media_type="application/json")
```